```
    if current != running:
        return current, 1
    else:
        return running, running_count
+ 1

def walk_rows_for_connect4(board):
    # walk over each row
    for i in range(len(board)):
        c, count = None, 0
        ## inner loop to look for
4-connected in a row
        for j in range(len(board[0])):
            c, count = compare_
element(board[i][j], c, count)
            if count == 4 and c != 'b':
                return (c, i, j,)
    return (None, (-1, -1))
```

The above should appear fairly intuitive to most programmers. The *walk_rows_for_connect4* function implements nested loops – the outer loop to walk over all the rows and the inner one to walk through elements of a row. In the inner loop, we keep track of a run, i.e., a contiguous sequence of same coloured chips. Once a run of length four is found, the function returns with the colour and the position. If none is found, it indicates failure by returning 'None' as the colour.

Implementing Step 2 in our algorithm (to handle columns) should also be similar. Let's scratch some code for it:

```
def walk_cols_for_connect4(board):
    # walk each column
    for i in range(len(board[0])):
        c, count = None, 0
        ## inner loop to look for
4-connected within a column
        for j in range(len(board
            c, count = compare_
element(board[j][i], c, count)
            if count == 4 and c != 'b':
                return (c, i, j,)
    return (None, -1, -1,)
```

The reader will observe that the above code for *walk_cols_for_connect4* has a significant overlap with its rows' counterpart. The reader might also note that both a row and column represent a sequence (or slice) on which *connect4* must be checked. Thus, in the next version, we employ a 'slice walker' that is called by both the row and column functions.

```
# compare_element not repeated for
brevity

def walk_slices_for_connect4(board,
outer, inner, is_reversed=False):
    for i in range(outer):
        c, count = None, 0
        ## inner loop to look for
4-connected in a slice
        for j in range(inner):
            v = board[j][i] if is_
reversed else board[i][j]
            c, count = compare_
element(v, c, count)
            if count == 4 and c != 'b':
                return (c, i, j,)
    return (None, -1, -1,)

def walk_rows_for_connect4(board):
    return walk_slices_for_
connect4(board, len(board),
len(board[0]))

def walk_cols_for_connect4(board):
    return walk_slices_for_
connect4(board, len(board[0]),
len(board),
                                is_
reversed=True)
```

While duplicated code was mostly eliminated, it was necessary to add the *is_reversed* flag, as the order of indices when walking rows and columns is different. This flag does stick out as a little ugly, but is needed to manage the order of the indices. It is easier to explain this with an example.

To walk the first row, the board elements to be checked are:

```
Board[0][0], board[0][1], board[0][2],
board[0][3], etc.
```

Walking the first column entails the following sequence of elements:

```
Board[0][0], board[1][0], board[2][0],
board[3][0] and so on.
```

For now, the flag serves this purpose and we move on. (We will look at it again at the end of this section.)

As we further analyse the *walk_slices_for_connect4* function, we observe that it performs three activities:

1. Enumerates all slices – driven by the 'outer' variable
2. Walks through one slice – driven by the 'inner' variable
3. Looks for four connected chips in a slice

In the most recent version, the slice walker handles both the Steps 1 and 2 above. To match the modularity of the code with the described steps, the final version of this section is arrived at, as follows:

```
def walk_a_slice_for_connect4(board,
outer_index, inner,
                              is_
reversed=False):
    c, count, i = None, 0, outer_index
    for j in range(inner): ## Step 2
        v = board[j][i] if is_reversed
else board[i][j]
        c, count = compare_element(v, c,
count)
        if count == 4 and c != 'b':
            return (c, i, j,)
    return (None, -1, -1,)

def walk_slices_for_connect4(board,
outer, inner,
                             is_
reversed=False):
    for i in range(outer):    ## Step 1
        c, x, y = walk_a_slice_for_
connect4(board, i, inner,

is_reversed)
        if c:
            return (c, x, y)

    return (None, -1, -1,)
```